

Traductor de Lenguaje Natural a Emojis

Proyecto Integrador Semestral — Programación de Sistemas de Base 2

Semestre	2026-1	Materia	Prog. de Sistemas de Base 2
Institución	Universidad Autónoma de Tamaulipas	Fecha	Mayo 2026
Profesor	Muñoz Quintero Dante Adolfo	Repo	Equipo Emojis

INTEGRANTES DEL EQUIPO

Cabrera Gabriel Cenyz Daylaan	a22133332138
López Alavez Gloria Jazmín	a2213332173
Huerta Hernández Dilan Dariel	a2213332171

TABLA DE CONTENIDO

1	Introducción	
2	Objetivos	
3	Especificación del Lenguaje	
	■ Categorías de Token	
	■ Construcciones del Lenguaje	
4	Arquitectura del Compilador	
5	Implementación	
	■ 5.1 Gramática del Lenguaje	
	■ 5.2 Análisis Semántico y Tabla de Símbolos	
6	Demostración Funcional	
	■ 6.1 Ejecución con programa válido	
	■ 6.2 Errores sintácticos detectados	
	■ 6.3 Errores semánticos y de tipo	
7	Conclusiones	

INTRODUCCIÓN

En la era digital, la comunicación se ha vuelto cada vez más visual y rápida, y los emojis se han consolidado como un medio universal para expresar emociones, ideas y conceptos de manera concisa. Sin embargo, a pesar de su popularidad, la traducción automática de texto en lenguaje natural a emojis no es trivial, ya que requiere comprender no solo el significado de las palabras individuales, sino también la estructura gramatical y el contexto de las oraciones.

El presente proyecto tiene como objetivo desarrollar un traductor de lenguaje natural a emojis, aplicando técnicas fundamentales del procesamiento de lenguajes y compiladores, como el análisis léxico, sintáctico, semántico. La herramienta identificará tokens léxicos, analizará la estructura de las oraciones y transformará el texto en secuencias de emojis que mantengan el significado y la intención original. Este enfoque permitirá demostrar la utilidad práctica de los conceptos de compilación y procesamiento de lenguajes en un problema moderno y cotidiano, facilitando la comunicación y ofreciendo un ejemplo claro de cómo las técnicas de análisis de texto pueden adaptarse a nuevas formas de expresión.

2

OBJETIVO

Diseñar y desarrollar una herramienta que traduzca texto en lenguaje natural a secuencias de emojis, aplicando de manera práctica las técnicas de análisis léxico, sintáctico y semántico estudiadas en la materia. El sistema deberá ser capaz de identificar tokens léxicos, analizar la estructura gramatical de las oraciones y mapear palabras o frases a los emojis más representativos según reglas definidas previamente. Este proyecto permitirá demostrar cómo las etapas fundamentales de un compilador (lexicalización, análisis sintáctico y transformación de información) pueden aplicarse a un problema práctico de procesamiento de lenguaje, facilitando la interpretación y comunicación de ideas a través de representaciones visuales.

ESPECIFICACIÓN DEL LENGUAJE

3.1 Categorías de Token

EMOCIONES	ACCIONES	LUGARES
FELIZ 	COMIENDO 	PLAYA 
TRISTE 	TRABAJANDO 	CASA 
ENOJADO 	ESTUDIANDO 	ESCUELA 
ESTADOS	CORRIENDO 	
HAMBRE 	OBJETOS	
CANSADO 	PIZZA 	
SUEÑO 	CAFÉ 	
	MÚSICA 	

3.2 Construcciones del Lenguaje

Declaración de función con llamada (reglas semánticas):

```
sujetos = ["estoy", "voy", "tengo"]

reglas_semanticas = {
  "estoy": ["EMOCION", "ESTADO", "ACCION"],
  "voy": ["ACCION", "LUGAR"],
  "tengo": ["ESTADO", "OBJETO"]
}
```

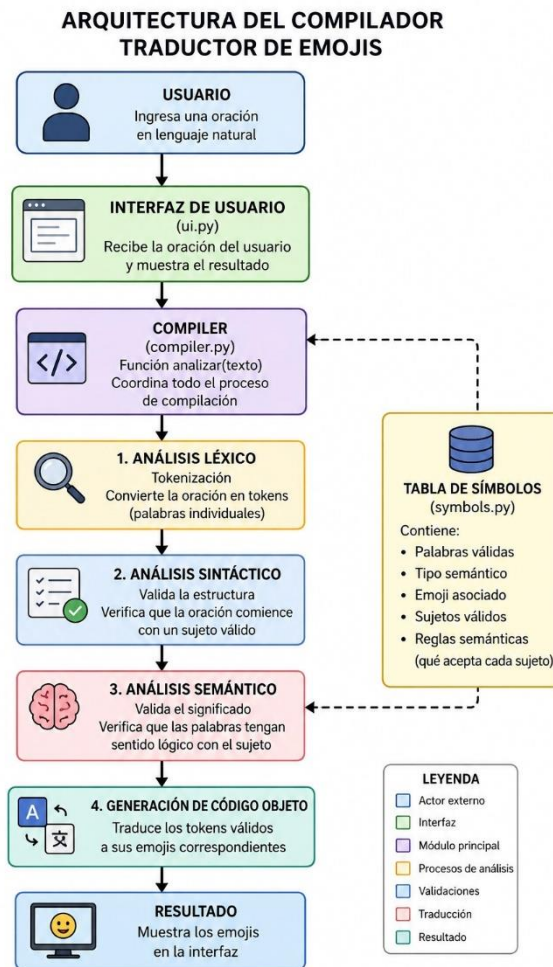
Aquí se pueden observar las reglas semánticas usadas en este traductor, donde:

Emoción: Contiene palabras relacionadas con emociones o sentimientos

Acción: Contiene verbos en gerundio que representan actividades

Estado: Incluye condiciones físicas o situaciones personales

ARQUITECTURA DEL COMPILADOR



La arquitectura del compilador “Traductor de Emojis”, desarrollado en Python mediante módulos organizados que trabajan de manera secuencial para analizar una oración escrita por el usuario y convertirla en emojis válidos. El proceso comienza cuando el usuario ingresa una frase en lenguaje natural desde la interfaz gráfica creada con Tkinter en el archivo ui.py. Esta interfaz recibe el texto, ejecuta el análisis y muestra el resultado final en pantalla.

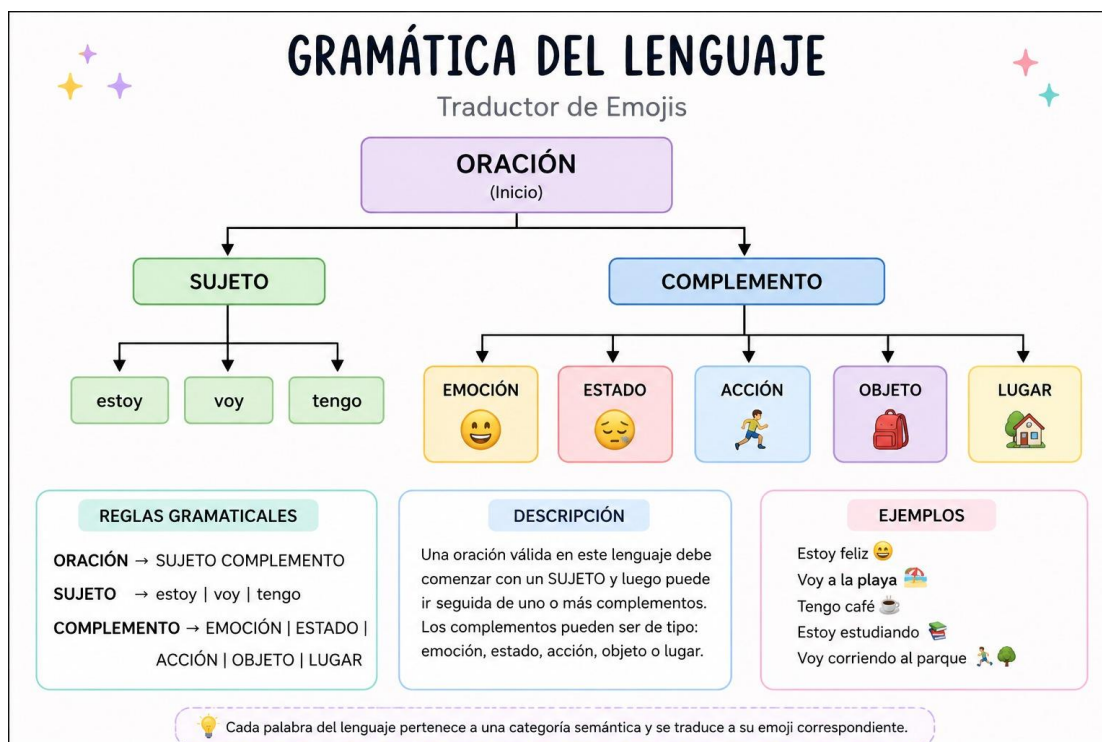
El núcleo principal del sistema se encuentra en el archivo compiler.py, específicamente en la función `analizar(texto)`, encargada de coordinar todas las etapas del proceso de compilación. Primero se realiza el análisis léxico, donde la oración es dividida en tokens o palabras individuales mediante el método `split()`. Posteriormente, el análisis sintáctico verifica que la estructura de la oración sea válida y que inicie con un sujeto permitido como “estoy”, “voy” o “tengo”.

Después se ejecuta el análisis semántico, utilizando las reglas almacenadas en la tabla de símbolos (symbols.py). En esta tabla se encuentran las palabras válidas del sistema, su tipo semántico (EMOCIÓN, ESTADO, ACCIÓN, OBJETO o LUGAR), el emoji correspondiente y las reglas que determinan qué tipos de palabras acepta cada sujeto. Por ejemplo, el sujeto “estoy” acepta emociones, estados y acciones, mientras que “voy” acepta acciones y lugares.

Si la oración cumple correctamente las reglas sintácticas y semánticas, el compilador pasa a la generación de código objeto, donde las palabras válidas son traducidas a sus emojis correspondientes. Finalmente, el resultado es mostrado en la interfaz gráfica, permitiendo al usuario visualizar la traducción realizada por el compilador.

IMPLEMENTACIÓN

5.1 Gramática del Lenguaje



5.2 Análisis Semántico y Tabla de Símbolos

El traductor reconoce palabras organizadas por categorías semánticas

```
1  tabla_smbolos = {
2      "feliz": ("EMOCION", "😊"),
3      "triste": ("EMOCION", "😞"),
4      "enojado": ("EMOCION", "😡"),
5      "emocionado": ("EMOCION", "😄"),
6      "asustado": ("EMOCION", "😱"),
7      "sorprendido": ("EMOCION", "😲"),
8      "nervioso": ("EMOCION", "😰"),
9      "relajado": ("EMOCION", "😌"),
10     "confundido": ("EMOCION", "😵"),
11     "aburrido": ("EMOCION", "😴"),
12
13     "hambre": ("ESTADO", "😋"),
14     "cansado": ("ESTADO", "😫"),
15     "sueño": ("ESTADO", "😴"),
16     "enfermo": ("ESTADO", "😷"),
17     "ocupado": ("ESTADO", "📅"),
18     "despierto": ("ESTADO", "🕒"),
19     "activo": ("ESTADO", "🚶"),
20     "caliente": ("ESTADO", "🔥"),
21     "frio": ("ESTADO", "❄️"),
22     "solo": ("ESTADO", "🌲"),
23
24     "comiendo": ("ACCION", "🍴"),
25     "trabajando": ("ACCION", "💼"),
26     "estudiando": ("ACCION", "📖"),
27     "corriendo": ("ACCION", "🏃"),
28     "durmiento": ("ACCION", "😴"),
29     "lavando": ("ACCION", "🧼"),
30     "escribiendo": ("ACCION", "🖋️"),
31     "cantando": ("ACCION", "🎤"),
32     "bailando": ("ACCION", "💃"),
33     "jugando": ("ACCION", "🎮"),
34     "viajando": ("ACCION", "🛩️),
35     "comunicando": ("ACCION", "📞)
```

En la imagen se puede observar la tabla de símbolos que permite identificar estados emocionales de una persona, cada emoción tiene un emoji asociado para representar visualmente el significado, describe cómo se encuentra una persona físicamente o en qué condición esta, representa acciones que una persona está realizando.

```
import tkinter as tk
from tkinter import messagebox
from compiler import analizer

def iniciar_app():
    ventana = tk.Tk()
    ventana.title("TRADUCTOR DE EMOJIS")
    ventana.geometry("900x600")
    ventana.config(bg="#1e1e1e")

    def compilar():
        texto = entrada.get().strip()

        if texto == "":
            messagebox.showwarning(title="Advertencia", message="Debes escribir texto")
            return

        resultado_final = analizer(texto)
        resultado.config(text=resultado_final)

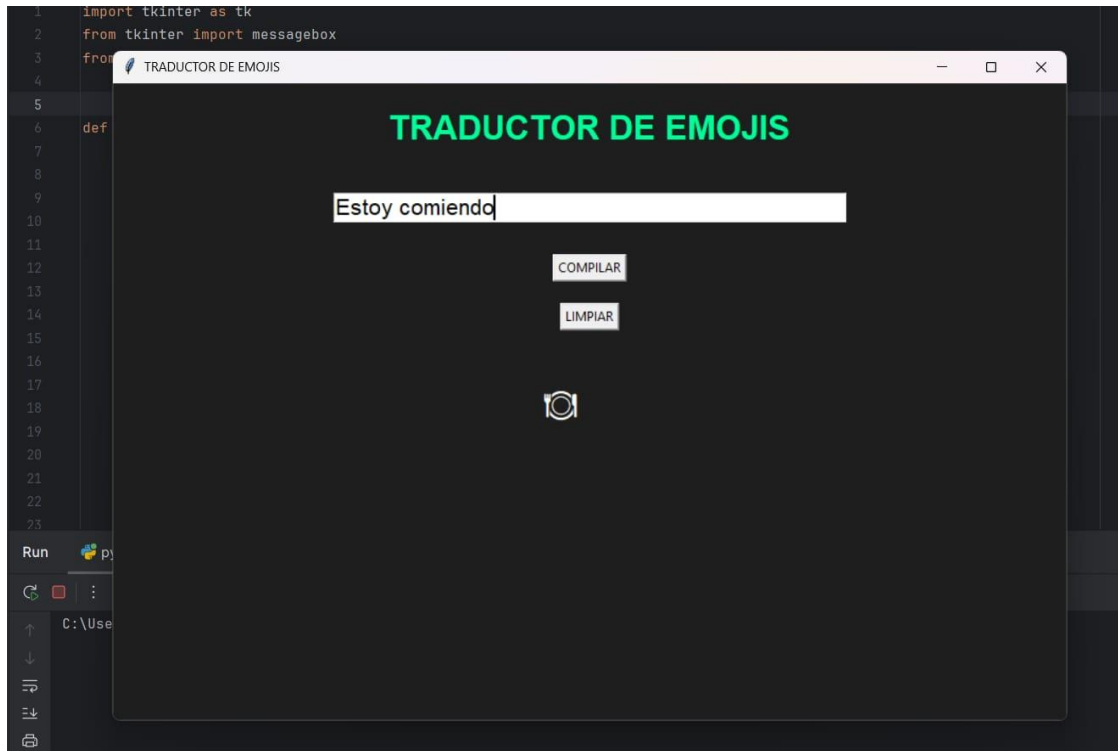
    def limpiar():
        entrada.delete(0, tk.END)
        resultado.config(text="")

    titulo = tk.Label(
        ventana,
        text="TRADUCTOR DE EMOJIS",
        font=("Arial", 24, "bold"),
        bg="#1e1e1e",
        fg="#00ff99"
    )
    titulo.pack(pady=20)

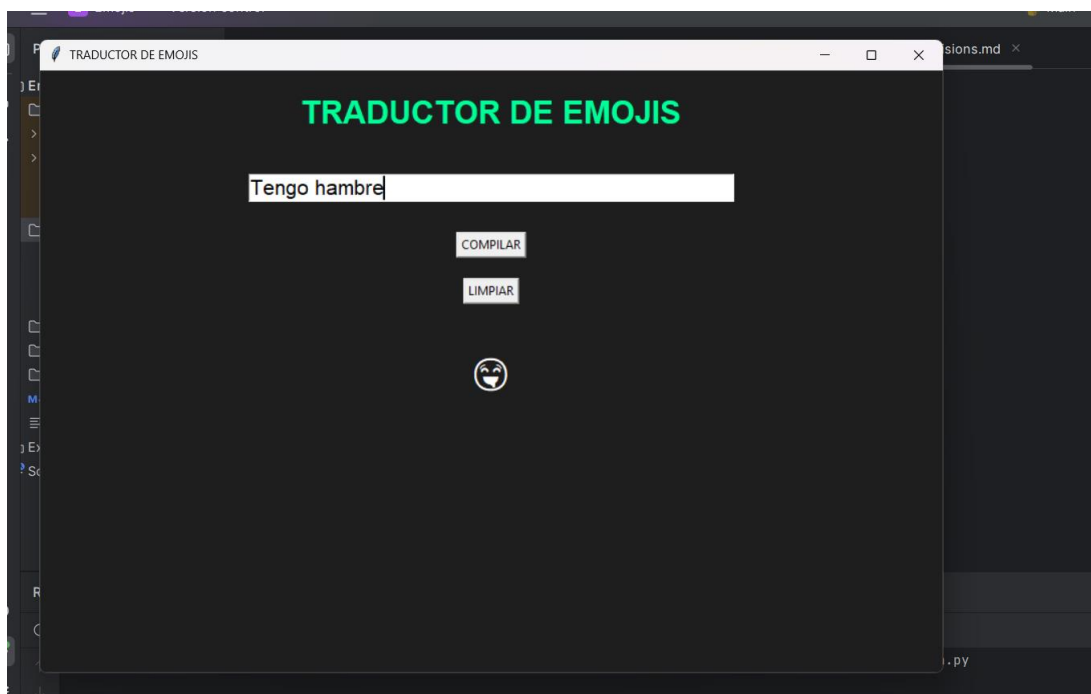
    entrada = tk.Entry(
```

DEMOSTRACIÓN FUNCIONAL

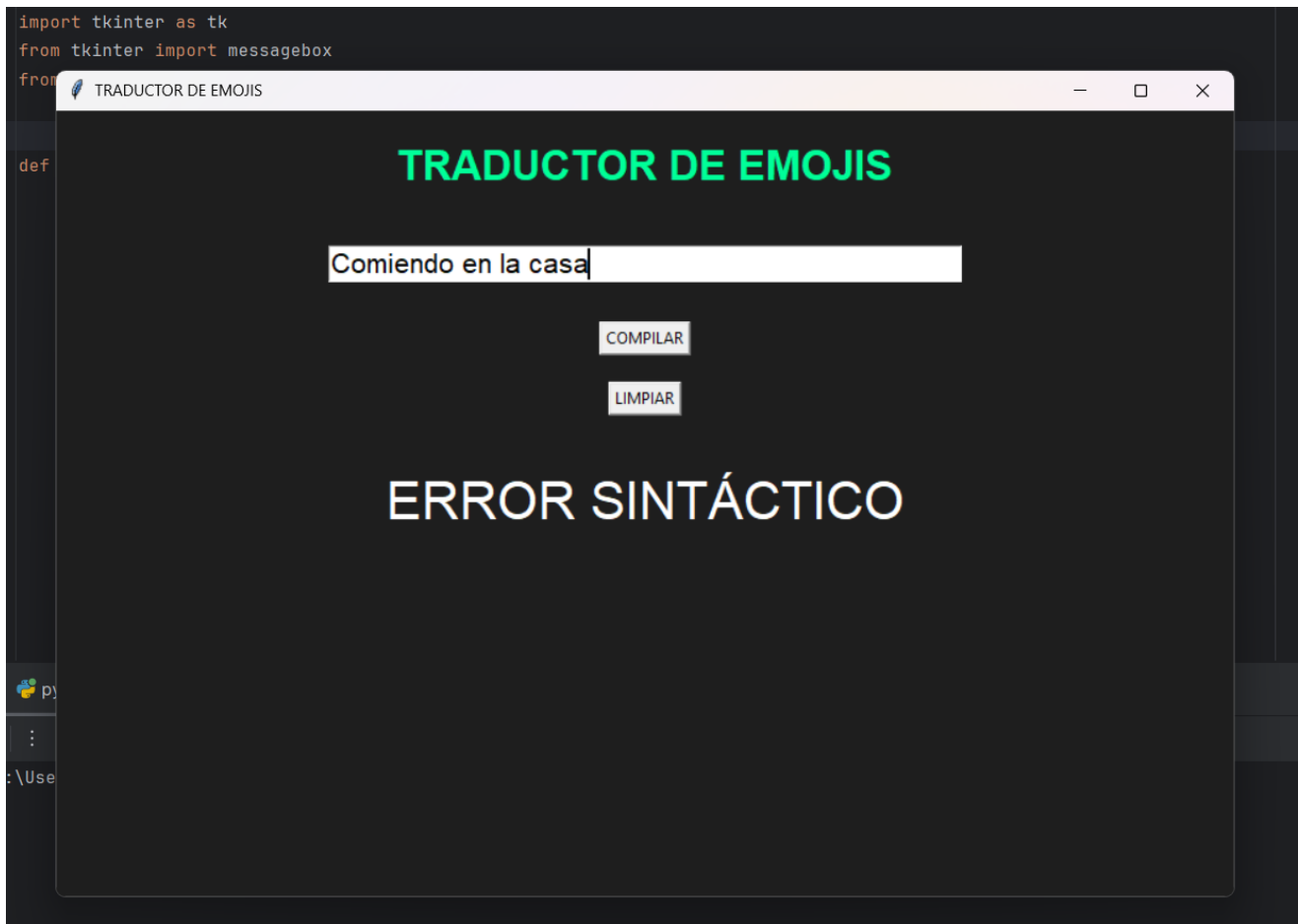
6.1 Ejecución con programa válido



Aquí se muestra la ejecución del traductor con las oraciones permitidas, con forme a la regla semántica proporcionada en el código.

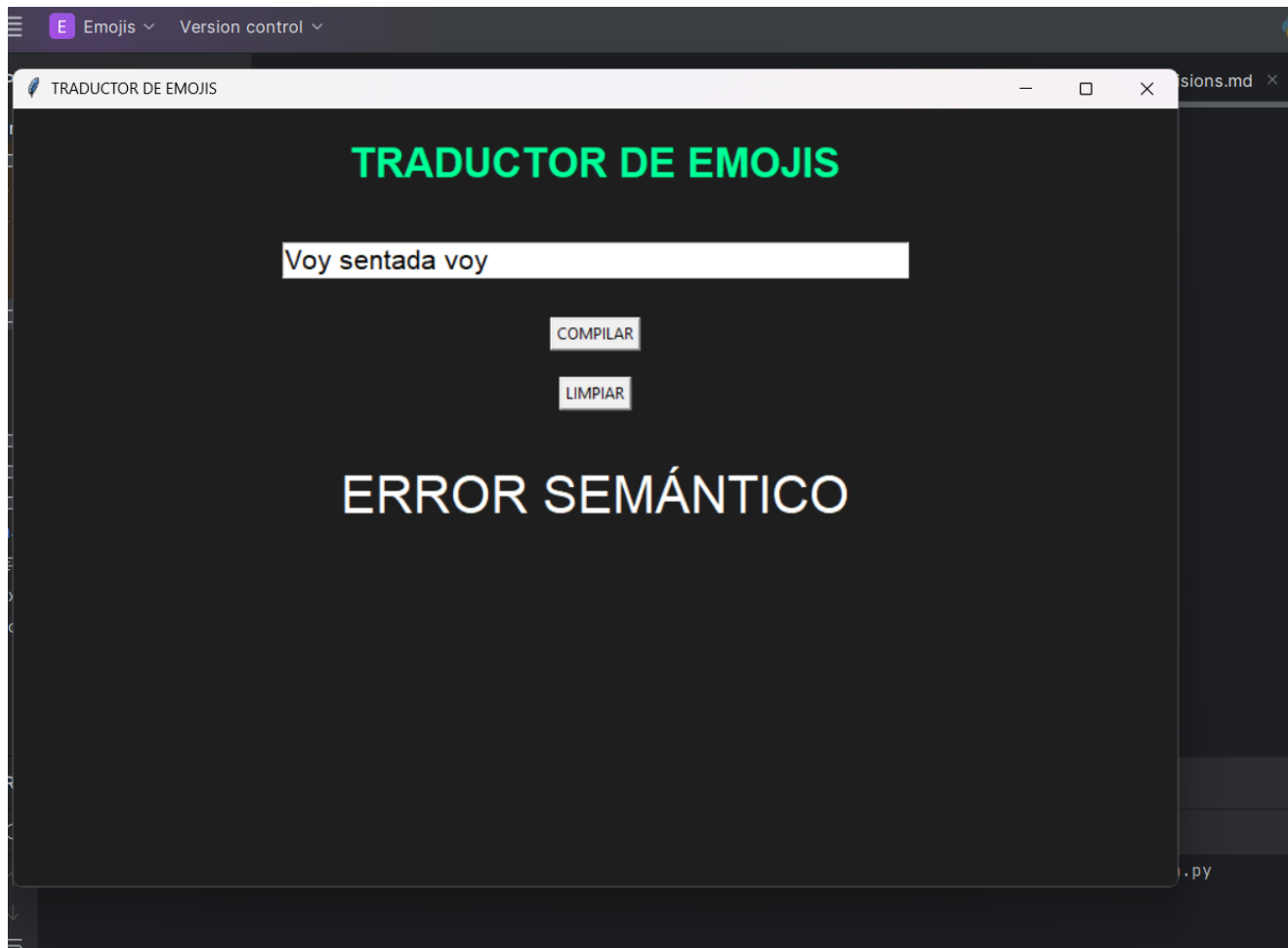


6.2 Error Sintáctico Detectado



Aquí se muestra el error sintáctico de manera en que las palabras no aprobadas por la regla semántica serían (en, la)

6.2 Error Semántico Detectado



En esta parte se puede mostrar un error semántico ya que, la oración no tiene coherencia en ella, por lo tanto, no se reconoce.

COLCUSIÓN

El desarrollo de un traductor de lenguaje natural a emojis demuestra cómo es posible combinar técnicas de procesamiento de lenguaje, gramáticas formales y estructuras de datos para transformar texto común en un lenguaje visual más expresivo y universal. A través del uso de herramientas como PYTHON, el sistema logra analizar oraciones, identificar palabras clave y aplicar reglas semánticas que permiten mapear conceptos a símbolos gráficos.

Este tipo de traductores no solo facilita una comunicación más dinámica y atractiva en entornos digitales, sino que también evidencia cómo los emojis pueden funcionar como un puente simbólico que simplifica ideas y emociones. Además, el proyecto muestra la importancia de definir diccionarios personalizados y reglas bien diseñadas para obtener traducciones coherentes y contextualizadas.

En conjunto, el traductor representa una aplicación práctica del procesamiento de lenguaje natural, donde se integran análisis léxico, sintáctico y semántico para producir un resultado comprensible, creativo y accesible para cualquier usuario. Es un ejemplo funcional de cómo la tecnología puede acercarse a formas modernas de comunicación y adaptarse a nuevas formas de expresión digital.